

NOMBRE DE LA UNIVERSIDAD

MAESTRÍA EN INGENIERÍA DE SOFTWARE

Desarrollo de Aplicaciones Empresariales

Sistema de Reserva de Canchas Deportivas

Arquitectura de Microfrontends y Microservicios

Proyecto Integrador

Autores

Pablo Jara Muñoz

pablojaramunoz@gmail.com

Miguel Lara

miguel.larasj@gmail.com

Sebastián Uyaguari

sebastianuyaguari@gmail.com

Adrián Espinoza

adrian.espinoza1292@gmail.com

Docente

Nombre del Docente

Ciudad, 25 de agosto de 2026

Resumen

Palabras clave:

Índice

Resumen	i
1. Introducción	1
1.1. Planteamiento del problema	1
1.2. Justificación	1
1.3. Estructura del documento	1
2. Objetivos	1
2.1. Objetivo general	1
2.2. Objetivos específicos	1
3. Alcance funcional	1
3.1. Roles y permisos	1
3.2. Módulos y pantallas	2
3.3. Fuera de alcance	2
4. Arquitectura de la solución	2
4.1. Visión general	2
4.2. Notación	3
4.3. Vistas arquitectónicas	3
4.4. Seguridad y control de acceso	8
4.5. Registro de decisiones de arquitectura	8
5. Modelo de datos	10
5.1. Estrategia de persistencia	10
5.2. Diagrama entidad-relación	11
5.3. Diccionario de datos	12
5.4. Integridad entre bases de datos	12
5.5. Restricciones que implementan reglas de negocio	13

6. Reglas de negocio	14
6.1. Catálogo de reglas	14
6.2. Validación de solapamiento de horarios	16
6.3. Estados de una reserva	17
6.4. Traducción de reglas a respuestas HTTP	18
7. Resultados	19
7.1. Funcionalidades implementadas	19
7.2. Cumplimiento de los criterios de aceptación	20
7.3. Verificación de las reglas de negocio	20
7.4. Limitaciones	21
8. Conclusiones	21
8.1. Lecciones aprendidas	21
8.2. Trabajo futuro	21
Referencias	25
A. Scripts de base de datos	25
B. Colección de pruebas de la API	25
C. Capturas de la aplicación	25
D. Repositorio del proyecto	25

Índice de figuras

1.	Vista de contexto: el sistema y sus dos tipos de usuario.	4
5.	Modelo entidad-relación, con la frontera de cada base de datos.	11
2.	Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.	22
3.	Vista de componentes de ms-reservas	23
4.	Vista de despliegue: contenedores Docker, redes y volumen.	24

Índice de tablas

1.	Matriz de roles y permisos.	1
2.	Módulos y pantallas del sistema.	2
3.	Composición de microfrontends.	5
4.	Composición de microservicios.	6
5.	Reparto de bases de datos.	11
6.	Entidad reservas	12
7.	Restricciones e índices por tabla.	14
8.	Reglas de negocio y dónde se implementa cada una.	15
9.	Correspondencia entre violación de regla y respuesta HTTP.	19
10.	Cumplimiento de los criterios de aceptación.	20
11.	Comprobación de las reglas de negocio.	21

Pendientes

1. Introducción

1.1. Planteamiento del problema

kfñlaksjdñflkjasñdlkfj dfoiajsdñlikfja sckfjañsldkfja asdkljfñalskdjflñ
aklsdjfñlkajsd

1.2. Justificación

1.3. Estructura del documento

2. Objetivos

2.1. Objetivo general

2.2. Objetivos específicos

-
-
-

3. Alcance funcional

3.1. Roles y permisos

Instrucción. Una matriz de funcionalidad por rol se lee de un vistazo y evita párrafos repetitivos. Marca con «Sí» y «No»; si algún permiso depende de una condición (cancelar solo las propias), dilo en la celda.

Tabla 1: Matriz de roles y permisos.

Funcionalidad	Usuario final	Administrador
[...]	[...]	[...]

3.2. Módulos y pantallas

Instrucción. *Los módulos funcionales con sus pantallas. Conviene que esta tabla case con los microfrontends de la Tabla 3: si un módulo no corresponde a ningún remote, o al revés, es señal de que la descomposición no está clara.*

Tabla 2: Módulos y pantallas del sistema.

Módulo	Pantalla	Descripción
[...]	[...]	[...]

3.3. Fuera de alcance

Instrucción. *Enumera lo excluido. No es una disculpa: delimitar el alcance es una decisión de diseño, y declararla evita que se lea como una carencia.*

[Funcionalidades excluidas y el motivo.]

4. Arquitectura de la solución

Instrucción. *Esta sección responde a una sola pregunta: por qué el sistema está partido así y no de otra manera. Describir los componentes es la parte fácil; lo que se evalúa es la justificación. Apóyate en las vistas para mostrar la estructura y en el registro de decisiones para explicarla.*

4.1. Visión general

El sistema aplica **microservicios en el backend y microfrontends en el frontend**, con un **edge** nginx delante que unifica el origen y un API gateway que es el único camino del navegador hacia los servicios. Son once piezas desplegables: cuatro paquetes de frontend, cinco servicios de backend —cuatro de dominio más el gateway—, el edge y una instancia de PostgreSQL con tres bases independientes.

El principio que gobierna el reparto es la **propiedad de los datos**: cada servicio es dueño de un dominio y de la base que lo sostiene, y nadie más la lee. De ahí salen las fronteras, no al revés. **ms-reservas** necesita el horario de una cancha y no lo consulta en

`canchas_db`: se lo pregunta a `ms-canchas`. Esa restricción es la que obliga a que cada servicio tenga una responsabilidad completa en lugar de ser una capa de otro.

Dentro de cada microservicio de dominio la estructura es **hexagonal**: el modelo y los casos de uso definen puertos, y los adaptadores —web, JPA, cliente HTTP— dependen de ellos y nunca al revés. Es lo que permite probar las ocho reglas de negocio con el puerto de salida sustituido por un doble, sin levantar Spring ni una base de datos. El gateway es la excepción deliberada: es infraestructura transversal, sin dominio propio que proteger, y su estructura se queda en `AuthenticationFilter` → `RouteConfig` → `ErrorHandler`.

4.2. Notación

Los diagramas siguen el modelo C4 [1]: contexto, contenedores y componentes, más una vista de despliegue. Están generados desde una única descripción en Structurizr DSL (`diagramas/workspace.dsl`), de modo que las cuatro vistas se mantienen consistentes entre sí por construcción.

4.3. Vistas arquitectónicas

Instrucción. Una subsección por vista. En cada una: el diagrama, y debajo la prosa que explica lo que el diagrama no puede decir — por qué esa frontera, qué pasa si un elemento falla, qué se decidió a propósito. Un diagrama sin texto no comunica; un texto que solo enumera lo que ya se ve en el diagrama, tampoco.

4.3.1. V-01 Contexto del sistema

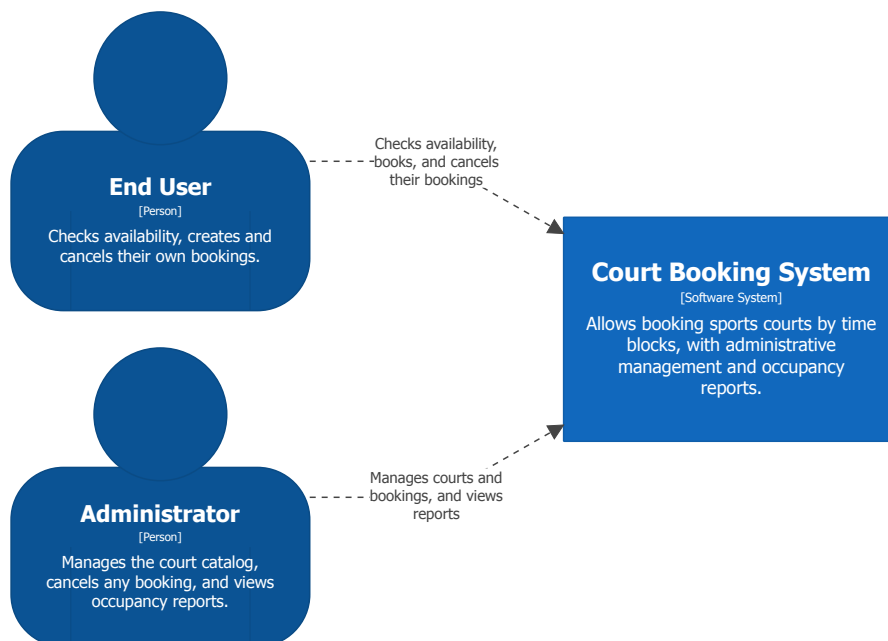


Figura 1: Vista de contexto: el sistema y sus dos tipos de usuario.

El sistema resuelve la reserva de canchas deportivas de un club para dos tipos de usuario. El **usuario final** consulta la disponibilidad de una cancha en una fecha, reserva un bloque libre, ve sus reservas y las cancela mientras no hayan empezado. El **administrador** gestiona el catálogo de canchas y sus horarios, cancela cualquier reserva y consulta los indicadores de ocupación. No hay más actores ni integraciones externas: no existe pasarela de pagos ni envío de notificaciones, que están fuera del alcance.

4.3.2. V-02 Contenedores

Es la vista principal del documento: contiene a la vez la composición de microfrontends y la de microservicios.

Frontend. Cuatro paquetes npm independientes, integrados en tiempo de ejecución con Module Federation. El **shell** es el host: enruta, mantiene la sesión y carga cada

remote de forma perezosa, solo cuando el usuario entra en su ruta. Cada remote expone únicamente `./App` y se construye y despliega por separado, sin depender del ciclo de vida de los demás.

Lo único que cruza la frontera entre paquetes es la prop `session` que baja el shell y los chunks compartidos de React. No hay imports entre remotes: si los hubiera, dejarían de ser desplegables por separado y el estilo se quedaría en el nombre.

Tabla 3: Composición de microfrontends.

Microfrontend	Responsabilidad	Tipo
<code>shell</code>	Enrutado, sesión, login y registro, y carga de los remotes.	Host
<code>mf-reservas</code>	Disponibilidad, nueva reserva y «mis reservas» del usuario final.	Remote
<code>mf-administracion</code>	Gestión de canchas, de reservas y de usuarios.	Remote
<code>mf-reportes</code>	Los cuatro indicadores de ocupación y demanda.	Remote

Backend. Cuatro microservicios de dominio y un gateway. Tres de los cuatro tienen base propia; `ms-reportes` **no tiene ninguna**, y es deliberado: un reporte de ocupación necesita cruzar reservas con canchas, y la forma cómoda de hacerlo sería leer las dos bases. Eso rompería la propiedad de los datos por conveniencia de una consulta. En su lugar agrega vía REST, llamando a `ms-reservas` y `ms-canchas` por puertos de salida. Ni siquiera tiene el driver JDBC en su `pom.xml`, de modo que la tentación no está disponible.

Tabla 4: Composición de microservicios.

Microservicio	Responsabilidad	Base de datos
<code>ms-usuarios</code>	Registro, autenticación y gestión de usuarios y roles.	<code>usuarios_db</code>
<code>ms-canchas</code>	Catálogo de canchas, horarios de atención y bloqueos de mantenimiento.	<code>canchas_db</code>
<code>ms-reservas</code>	Reservas y disponibilidad. Dueño de RN-01–RN-06 y RN-08 .	<code>reservas_db</code>
<code>ms-reportes</code>	Indicadores de ocupación y demanda; agrega vía REST.	ninguna

Comunicación. Todo es HTTP con JSON, síncrono. El navegador solo conoce `http://localhost:` el edge sirve el shell en la raíz, cada remote bajo su prefijo, y reenvía `/api/` al gateway. El edge es el **único contenedor conectado a las dos redes** de Docker, y eso es lo que impide que el navegador alcance un microservicio saltándose el gateway.

Entre servicios hay dos llamadas, y las dos existen para no leer una base ajena:

- `gateway` → `ms-usuarios`, para verificar la credencial de cada petición.
- `ms-reservas` → `ms-canchas`, para comprobar que la cancha existe, está activa y admite el bloque antes de persistir.
- `ms-reportes` → `ms-reservas` y `ms-canchas`, para agregar los indicadores.

Si un destino no responde, el fallo se traduce a **503** y no a un 500 genérico: la diferencia es la que permite que la pantalla diga «inténtalo de nuevo» en lugar de «algo se rompió». En `ms-reservas` eso significa que una reserva no se crea si no se pudo validar la cancha; persistirla y reconciliar después dejaría reservas sobre canchas inexistentes.

4.3.3. V-03 Componentes de `ms-reservas`

Se detalla este servicio y no otro porque es donde viven las reglas de negocio del sistema (Apartado 6).

Una petición `POST /api/reservas` recorre el hexágono en este orden, y cada salto tiene un motivo:

1. `BookingController` (*adaptador de entrada*) traduce HTTP a DTO y lee la identidad de `X-User-Id` y `X-User-Role`. No decide nada: no hay en él un solo `if` de negocio.
2. `BookingUseCase` (*puerto de entrada*) es lo único que el controlador conoce. No sabe qué implementación hay detrás.
3. `BookingService` (*aplicación*) aplica las reglas en orden: **FR-018** —un administrador no reserva—, después la cancha vía `CourtClientPort`, después **RN-06** con el tope leído por `ConfigurationRepositoryPort`, y por último **RN-02**. El orden importa: si el usuario ya llegó al tope, el bloque concreto da igual y el mensaje es más útil.
4. `BookingRepositoryPort` (*puerto de salida*) recibe un `Booking` de dominio. El servicio no sabe que detrás hay JPA.
5. `BookingRepositoryAdapter` (*adaptador de salida*) traduce a la entidad JPA, persiste, y **captura la violación de la restricción `EXCLUDE` traduciéndola a `SlotAlreadyBookedException`**. Es el punto donde una excepción de PostgreSQL deja de existir para el dominio.

El reloj se inyecta en `BookingService` en lugar de leerse con `LocalDateTime.now()` allí donde haga falta. Parece un detalle y no lo es: **RN-04** y la derivación de `FINALIZADA` dependen del instante actual, y sin poder fijarlo una prueba tendría que dormir o depender de la hora a la que se ejecute.

4.3.4. V-04 Despliegue

Dos redes bridge. La red `frontend` contiene el shell y los tres remotes; la red `backend`, el gateway, los cuatro microservicios y PostgreSQL. El `edge` es el único contenedor con un pie en cada una, y por tanto el único camino entre ellas. Un contenedor de frontend no puede alcanzar `ms-reservas` aunque conozca su nombre: no comparten red.

Los datos persisten en un volumen `pgdata`. Los scripts de `infra/postgres/init/` —que crean las tres bases, sus roles sin permisos cruzados y la extensión `btree_gist`— se ejecutan una sola vez, cuando el volumen está vacío; por eso `docker compose down -v` es lo que fuerza un arranque limpio.

El orden de arranque se apoya en `depends_on` con `condition: service_healthy`, sobre el `health` de Actuator de cada servicio y `pg_isready` para PostgreSQL. Sin eso el arranque es una carrera: Flyway intentaría migrar contra una base que todavía no acepta conexiones.

4.4. Seguridad y control de acceso

La autenticación es **HTTP Basic verificada en cada petición por el gateway**. No se emite ningún token: no hay JWT, ni sesión de servidor, ni nada que renovar o revocar. El alcance del proyecto pide autenticación básica con roles, y añadir OAuth2 sería construir un mecanismo que ningún criterio evalúa.

El recorrido de una petición autenticada es:

1. El navegador manda **Authorization: Basic** en toda petición a `/api/**`, salvo las dos rutas públicas —`POST /api/auth/login` y `POST /api/auth/registro`—, que existen precisamente para obtener la credencial.
2. El `AuthenticationFilter` del gateway **descarta primero cualquier cabecera `X-User-*` que venga del cliente**, verifica la credencial contra `ms-usuarios` y, si es válida y la cuenta está activa, añade `X-User-Id` y `X-User-Role`.
3. El microservicio destino lee esas dos cabeceras y aplica **RN-03** y **RN-07** sobre ellas. No recibe nunca las credenciales del usuario y no vuelve a autenticar.

El orden del segundo punto es lo que hace segura la confianza entre el gateway y los microservicios. Un microservicio cree lo que dice `X-User-Id` precisamente porque nadie más puede escribirla: si el borrado no fuera lo primero que ocurre, cualquiera se declararía administrador mandando una cabecera. La prueba `AuthenticationFilterTest` fija ese comportamiento, y el escenario V5 del `quickstart` lo comprueba de extremo a extremo con `curl`.

Las contraseñas se guardan hasheadas con `BCrypt` y nunca en claro. `ms-usuarios` depende de `spring-security-crypto` —la librería de criptografía— y no de `spring-boot-starter-security`. No hay cadena de filtros de seguridad en ningún microservicio, porque quien autentica es el gateway.

4.5. Registro de decisiones de arquitectura

Instrucción. *Cada decisión relevante con su alternativa descartada. Una decisión sin alternativa no es una decisión: es una descripción. Este registro es lo que distingue un documento de arquitectura de un manual de usuario del propio sistema. Referencia las decisiones desde el texto con `\addrref{01}`.*

ADR-01 Restricción de exclusión en PostgreSQL para el solapamiento

Contexto. **RN-02** tiene que sostenerse también cuando dos personas reservan el mismo bloque simultáneamente. Una comprobación en el servicio deja una ventana entre la lectura y la escritura por la que caben dos peticiones a la vez.

Decisión. Se mantiene la comprobación previa en `BookingService`, por el mensaje claro, y se añade una restricción `EXCLUDE USING gist` parcial sobre las reservas confirmadas. `BookingRepositoryAdapter` traduce su violación a la misma excepción de dominio que la comprobación previa.

Alternativa descartada. Validar solo en Java, con un bloqueo pesimista sobre la cancha o una transacción `SERIALIZABLE`. Se descartó: lo primero serializa todas las reservas de una cancha aunque sean de días distintos, y lo segundo obliga a una política de reintentos en toda la aplicación para un caso que el motor ya sabe resolver en el índice.

Consecuencia. Se gana una garantía que no depende de que el código la recuerde, y un 409 idéntico por los dos caminos. Se paga con una dependencia de PostgreSQL y su extensión `btree_gist`: el esquema deja de ser portable a otro motor.

ADR-02 Una base por microservicio, con un rol sin permisos cruzados

Contexto. La independencia de datos entre microservicios se evalúa, y una convención que solo vive en la cabeza del equipo se rompe en cuanto alguien tiene prisa.

Decisión. Tres bases en una única instancia de PostgreSQL, cada una con un rol propietario al que se le concede `CONNECT` solo sobre la suya, revocándolo a `PUBLIC`. `ms-reportes` no recibe rol ni driver JDBC.

Alternativa descartada. Un esquema compartido con prefijos de tabla, o tres instancias separadas de PostgreSQL. Lo primero no impide el acceso cruzado, solo lo desaconseja; lo segundo triplica memoria y tiempo de arranque sin ganar nada verificable.

Consecuencia. Se gana que una violación del principio falle al conectar en lugar de pasar desapercibida en revisión. Se paga con una consulta imposible: cruzar reservas y canchas exige una llamada HTTP, no un JOIN.

ADR-03 Un edge que unifica el origen, en lugar de CORS por servicio

Contexto. Los microfrontends federados piden sus chunks en tiempo de ejecución. Con cada remote en su propio puerto, el navegador los ve como orígenes distintos y bloquea la carga.

Decisión. Un nginx delante que sirve el shell en la raíz, cada remote bajo su prefijo y reenvía `/api/` al gateway. Un solo origen para todo.

Alternativa descartada. Configurar cabeceras CORS permisivas en cada remote y en el gateway. Se descartó porque hay que repetirlo en cinco sitios y cada uno es una oportunidad de abrir de más.

Consecuencia. Se gana que el problema desaparezca en lugar de parchearse, y un único punto donde publicar el sistema. Se paga con una pieza más en el despliegue.

Instrucción. *Candidatas a decisión en este proyecto, por si ayudan a arrancar: una base de datos por microservicio frente a un esquema compartido; la restricción de exclusión en PostgreSQL frente a validar el solapamiento en Java; el gateway como punto único frente a que cada microfrontend llame directamente a los servicios; `ms-reportes` agregando por REST frente a leer las tablas de otros servicios; el edge que unifica el origen frente a configurar CORS en cada remote.*

5. Modelo de datos

5.1. Estrategia de persistencia

Instrucción. *Explica el reparto: qué base pertenece a qué servicio y qué principio se sigue. Si se optó por una sola instancia de PostgreSQL con varias bases en lugar de varias instancias, dilo aquí y justifícalo — es una pregunta previsible en la defensa.*

Cada microservicio con estado es dueño de su propia base y nadie más la lee. La regla no admite excepciones: si `ms-reservas` necesita el horario de una cancha, se lo pregunta a `ms-canchas` por HTTP; no abre una conexión a `canchas_db`.

La decisión operativa es una **única instancia de PostgreSQL 16 con tres bases independientes**, no tres instancias. Con tres instancias el aislamiento sería más fuerte, pero el sistema entero tiene que levantarse con `docker compose up` en la máquina de quien lo evalúa, y triplicar el motor cuesta memoria y tiempo de arranque sin ganar nada demostrable en la rúbrica.

Lo que sí se conserva de un despliegue con instancias separadas es la parte que importa: **un rol de base distinto por servicio, sin permisos cruzados**. Al inicializar el volumen se revoca `CONNECT` a `PUBLIC` sobre cada base y se concede solo a su dueño:

```
1 CREATE DATABASE usuarios_db OWNER usuarios_app;
2 REVOKE CONNECT ON DATABASE usuarios_db FROM PUBLIC;
3 GRANT CONNECT ON DATABASE usuarios_db TO usuarios_app;
```

Listing 1: Aislamiento por credenciales, en `infra/postgres/init/01-bases-y-roles.sql`.

La diferencia es verificable: si alguien escribiera en `ms-reservas` una consulta contra `canchas_db`, no obtendría datos de otro dominio sino un **FATAL: permission denied for database**. La independencia deja de ser una convención que hay que recordar y pasa a ser algo que el motor impone.

`ms-reportes` no recibe rol ni credencial: no tiene base, agrega por REST.

Tabla 5: Reparto de bases de datos.

Base	Servicio dueño	Contenido
<code>usuarios_db</code>	<code>ms-usuarios</code>	Tabla <code>usuario</code> : credenciales (hash BCrypt), rol y estado de activación.
<code>canchas_db</code>	<code>ms-canchas</code>	Tablas <code>cancha</code> y <code>bloqueo_mantenimiento</code> : catálogo, deporte, horario de atención y bloqueos.
<code>reservas_db</code>	<code>ms-reservas</code>	Tablas <code>reserva</code> y <code>configuracion</code> . Aloja la restricción de exclusión que implementa RN-02 .

5.2. Diagrama entidad-relación

Instrucción. *Un DER por base, o uno global con las fronteras entre bases marcadas. Lo segundo comunica mejor la independencia, que es justo lo que se evalúa.*

Falta figuras/modelo-datos.pdf

Exporta el diagrama desde Structurizr y déjalo en `figuras/` con ese nombre.

Figura 5: Modelo entidad-relación, con la frontera de cada base de datos.

5.3. Diccionario de datos

Instrucción. *Una tabla por entidad. No hace falta repetir todos los tipos SQL: basta el campo, su tipo lógico y qué significa. Los tipos exactos están en los scripts del anexo.*

Tabla 6: Entidad reservas.

Campo	Tipo	Descripción
id	bigint	identidad Clave primaria.
cancha_id	bigint	Cancha reservada. Sin clave foránea: vive en otra base (Apartado 5.4).
usuario_id	bigint	Dueño de la reserva. Sin clave foránea, por lo mismo.
fecha	date	Día del bloque reservado.
hora_inicio	time	Inicio del bloque, incluido.
hora_fin	time	Fin del bloque, excluido. CHECK hora_fin > hora_inicio.
estado	varchar(20)	CONFIRMADA, CANCELADA o FINALIZADA, con CHECK . En la tabla solo aparecen los dos primeros: FINALIZADA se deriva al leer.
creada_en	timestamp	Instante de creación, con DEFAULT now() .
cancelada_en	timestamp	Nulo salvo que se haya cancelado. Alimenta el reporte de cancelaciones por período.

5.4. Integridad entre bases de datos

Instrucción. *Es el apartado que más distingue un modelo de microservicios de un monolito con varias tablas. Explica qué referencias cruzan la frontera entre bases, por qué no pueden ser claves foráneas y dónde se valida entonces la integridad.*

Dos columnas de `reserva` referencian entidades de otras bases: `cancha_id` apunta a

`canchas_db.cancha` y `usuario_id` a `usuarios_db.usuario`. Ninguna de las dos es una clave foránea, y su ausencia no es un descuido: una clave foránea es una restricción *dentro de una base*, y estas dos tablas están en bases distintas a las que la conexión de `ms-reservas` ni siquiera puede acceder. No es que se haya decidido no declararlas: es que no existe forma de declararlas.

La integridad se valida entonces donde sí puede validarse, antes de persistir:

- **La cancha:** `BookingService` llama a `ms-canchas` por `CourtClientPort` y comprueba que existe, que está activa y que el bloque cae dentro de su horario de atención. Si no existe, la petición se rechaza con 404; si existe pero no admite la reserva, con 422.
- **El usuario:** no hace falta comprobarlo. El identificador llega en la cabecera `X-User-Id`, que el gateway escribe solo después de verificar la credencial contra `ms-usuarios`, y descarta cualquier `X-User-*` que venga del cliente. Un identificador inventado no puede llegar hasta aquí.

Si `ms-canchas` no responde, la reserva no se crea: se devuelve un 503. La alternativa —persistir sin validar y reconciliar después— dejaría reservas sobre canchas inexistentes o inactivas, que es exactamente el estado que la clave foránea existía para impedir.

Inactivar un usuario o una cancha no toca las reservas ya hechas (FR-032, FR-047): son de otro dominio, y esas bases no las conocen.

5.5. Restricciones que implementan reglas de negocio

Instrucción. *Aquí va la restricción de exclusión que impide el solapamiento. Es el punto técnico más fuerte del modelo de datos: explica por qué está en el motor y no en el código de la aplicación. El detalle de la regla como tal va en la Apartado 6.2; aquí solo el mecanismo de base de datos.*

La pieza principal es la restricción de exclusión de `reserva`, que impide el solapamiento incluso bajo peticiones concurrentes. Su definición y el razonamiento que la justifica están en la Apartado 6.2; aquí interesa el mecanismo: es un índice **GiST** sobre el par `(cancha_id, tsrange(fecha + hora_inicio, fecha + hora_fin, '[]'))`, con un predicado parcial que lo limita a las filas confirmadas. PostgreSQL rechaza cualquier inserción cuyo rango se solape con uno ya indexado para la misma cancha, y lo hace serializando en el índice, no en la aplicación. Requiere la extensión `btree_gist`, instalada al inicializar el volumen.

El resto de restricciones son `CHECK` que fijan las enumeraciones cerradas y los rangos

válidos, de modo que un valor inválido se rechace aunque el código lo dejara pasar:

Tabla 7: Restricciones e índices por tabla.

Tabla	Restricción o índice	Qué sostiene
usuario	UNIQUE (username)	FR-002: el duplicado es imposible, no solo improbable.
usuario	CHECK rol IN (...)	Enumeración cerrada de roles.
cancha	CHECK deporte IN (...)	Enumeración cerrada de deportes.
cancha	CHECK hora_cierre > hora_apertura	RN-07: un horario invertido no delimita bloques.
reserva	EXCLUDE USING gist (...)	RN-02 , incluido el acceso concurrente.
reserva	CHECK estado IN (...)	RN-08.
reserva	INDEX (usuario_id, fecha)	Consulta «mis reservas».
reserva	INDEX (cancha_id, fecha)	Consulta de disponibilidad.

El esquema lo versiona Flyway y Hibernate arranca con `ddl-auto: validate`: si el mapeo de una entidad y la tabla real divergen, el servicio no arranca, en lugar de fallar más tarde con una columna inesperada.

6. Reglas de negocio

6.1. Catálogo de reglas

Instrucción. Una fila por regla, con la columna de implementación rellena de verdad: «ms-reservas, ReservaService» dice algo; «se valida en el backend» no. Esta tabla es la que se mira para puntuar este criterio.

Tabla 8: Reglas de negocio y dónde se implementa cada una.

ID	Descripción	Implementación
RN-01	Una reserva se hace sobre una cancha, una fecha y un bloque horario predefinido de una hora.	<code>ms-reservas:</code> <code>BookingService.crear</code> , sobre el objeto de valor <code>TimeSlot</code> . Prueba: <code>BookingServiceCreacionTest</code> .
RN-02	No pueden existir dos reservas confirmadas solapadas sobre la misma cancha, ni siquiera bajo peticiones concurrentes.	Doble mecanismo: comprobación previa en <code>BookingService.crear</code> y restricción <code>EXCLUDE</code> en <code>reservas_db</code> (Apartado 6.2). Pruebas: <code>BookingServiceCreacionTest</code> y <code>ConcurrenciaReservaIT</code> .
RN-03	El usuario final cancela solo sus reservas; el administrador, cualquiera.	<code>ms-reservas:</code> <code>BookingService.cancelar</code> , sobre el rol recibido en <code>X-User-Role</code> . Prueba: <code>BookingServiceCancelacionTest</code> .
RN-04	No se cancela una reserva cuya hora de inicio ya ocurrió, tampoco siendo administrador.	<code>ms-reservas:</code> <code>Booking.sePuedeCancelarEn</code> , invocado desde <code>BookingService.cancelar</code> . Prueba: <code>BookingServiceCancelacionTest</code> .
RN-05	Cancelar pasa la reserva a <code>CANCELADA</code> y libera su bloque, sin borrar la fila.	<code>ms-reservas:</code> <code>Booking.cancelar</code> . El bloque se libera porque la restricción <code>EXCLUDE</code> solo alcanza a las confirmadas. Prueba: <code>BookingServiceCancelacionTest</code> .
RN-06	Un usuario final no supera un tope configurable de reservas activas (3 por defecto).	<code>ms-reservas:</code> <code>BookingService.verificarTope</code> , con el valor leído de la tabla <code>configuracion</code> por <code>ConfigurationRepositoryPort</code> .

6.2. Validación de solapamiento de horarios

Instrucción. *La regla que el documento de alcance destaca sobre las demás. Dedícale espacio: qué problema hay si dos personas reservan a la vez, dónde se resuelve y por qué ahí. Si la validación está en la base de datos y no en el servicio, explica el razonamiento: es un argumento de concurrencia, no de comodidad.*

El problema no es detectar un solapamiento, que es trivial, sino detectarlo cuando dos personas reservan *a la vez*. Una comprobación en el servicio —leer las reservas de esa cancha y ese día, ver que el bloque está libre, insertar— tiene una ventana entre la lectura y la escritura. Si dos peticiones la atraviesan simultáneamente, ambas leen un bloque libre y ambas insertan: la regla se viola sin que ningún `if` haya fallado.

Por eso **RN-02** se sostiene sobre **dos** mecanismos, y no por redundancia:

1. **Comprobación previa en BookingService**, que cubre el caso normal y es la que produce un mensaje útil: «el bloque 19:00–20:00 de la cancha 3 ya está reservado».
2. **Una restricción de exclusión en PostgreSQL**, que cubre el caso que la primera no puede cubrir. Es el motor quien serializa, y lo hace sobre el índice, no sobre el código de la aplicación.

```

1 ALTER TABLE reserva ADD CONSTRAINT reserva_sin_solape
2   EXCLUDE USING gist (
3     cancha_id WITH =,
4     tsrange(fecha + hora_inicio, fecha + hora_fin, '[)') WITH &&
5   ) WHERE (estado = 'CONFIRMADA');
```

Listing 2: Restricción que implementa **RN-02**, en `V1__reservas.sql`.

Tres detalles de esa definición no son negociables:

- `WHERE (estado = 'CONFIRMADA')` hace que solo las confirmadas compitan por el bloque. Es lo que permite que cancelar libere el horario (**RN-05**) sin borrar la fila, conservando la traza que pide **RN-08** y el dato que necesita el reporte de cancelaciones.
- `'[)'` —inicio incluido, fin excluido— evita que una reserva de 10:00–11:00 bloquee la de 11:00–12:00. Sin esto, los bloques contiguos dejarían de poder reservarse.
- El `tsrange` se construye sobre `fecha + hora_inicio` porque esa expresión es inmutable y por tanto indexable; con una variante con zona horaria, la creación del índice falla.

La restricción exige la extensión `btree_gist`, que se instala en `reservas_db` al inicializar el volumen: un índice GiST no sabe comparar un `bigint` con `=` sin ella.

Cuando la restricción salta, PostgreSQL devuelve una violación de integridad. `BookingRepositoryAdapt` la captura y la traduce a `SlotAlreadyBookedException`, la misma excepción que lanza la comprobación previa. El dominio nunca ve una excepción de base de datos —lo que violaría la regla de dependencia hexagonal— y el cliente recibe el mismo **409** con el mismo cuerpo por los dos caminos, sin poder distinguirlos.

La prueba `ConcurrenciaReservaIT` lanza dos hilos retenidos en una barrera que reservan el mismo bloque al soltarse, contra PostgreSQL real, y comprueba que exactamente uno confirma. Se repite diez veces, que es lo que pide el criterio **SC-004**.

6.3. Estados de una reserva

Una reserva nace **CONFIRMADA** y no hay forma de crearla en otro estado: el constructor de `Booking` que usa el caso de uso no admite el estado como parámetro. Desde ahí solo salen dos caminos, y ambos son terminales:

Desde		Hasta	Qué la provoca
CONFIRMADA	→	CANCELADA	Una acción del dueño o del administrador, sujeta a RN-03 y RN-04 .
CONFIRMADA	→	FINALIZADA	El paso del tiempo: la hora de fin del bloque ya ocurrió.

La segunda transición merece una explicación, porque no la produce ninguna acción de usuario. **FINALIZADA no se persiste nunca**: se deriva al leer, comparando la hora de fin con el instante actual (`Booking.estadoVigente`). La alternativa —un proceso periódico que recorra la tabla marcando filas— tiene un modo de fallo que esta no tiene: si el proceso no corre, el estado queda desactualizado y una reserva terminada sigue contando como activa para **RN-06**. Derivándolo, eso es imposible por construcción.

De ahí se siguen dos consecuencias que las pruebas fijan: una reserva ya terminada no cuenta como activa para el tope de **RN-06**, y no se puede cancelar (**FR-028**).

6.4. Traducción de reglas a respuestas HTTP

Instrucción. *Qué código devuelve la API cuando se viola cada regla. Importa más de lo que parece: un 500 genérico ante una reserva duplicada es un defecto, y un 409 con un mensaje claro es lo que permite que la interfaz reaccione bien.*

La traducción ocurre en un único punto por servicio, la clase `ErrorHandler` de `adapter/in/web/`.

Las excepciones de `domain/exception/` no conocen `HttpStatus` ni nada de `org.springframework.web`. si lo conocieran, el núcleo de negocio dependería del protocolo por el que se le habla, que es justo lo que la regla de dependencia hexagonal prohíbe.

La distinción entre **409** y **422** es deliberada y no cosmética. Un 409 es un conflicto con el estado actual que *puede desaparecer solo*: el bloque puede liberarse si alguien cancela. Un 422 es una petición que no va a funcionar por reintentarla: la cancha está inactiva o el bloque no existe en su horario. La pantalla reacciona distinto a cada uno —el primero invita a reintentar, el segundo a elegir otra cosa— y por eso no puede recibir el mismo código.

Tabla 9: Correspondencia entre violación de regla y respuesta HTTP.

Regla	HTTP	Excepción de dominio y significado
RN-02	409	<code>SlotAlreadyBookedException</code> — el bloque ya tiene una reserva confirmada. Llega por la comprobación previa o por la restricción <code>EXCLUDE</code> , con idéntico cuerpo.
RN-06	409	<code>ActiveBookingLimitExceededException</code> — el usuario alcanzó el tope de reservas activas.
RN-04	409	<code>PastBookingCancellationException</code> — la hora de inicio ya ocurrió.
RN-03	403	<code>ForbiddenOperationException</code> — un usuario final intenta cancelar una reserva ajena, o un administrador intenta crear una reserva (FR-018).
—	404	<code>BookingNotFoundException</code> o <code>CourtNotFoundException</code> — el recurso no existe.
—	422	<code>CourtNotBookableException</code> — la cancha existe pero no admite la reserva: está inactiva, el bloque cae fuera de su horario, o la fecha ya pasó.
—	400	Violación de <code>jakarta.validation</code> o valor fuera de una enumeración cerrada.

7. Resultados

7.1. Funcionalidades implementadas

Instrucción. *Qué quedó operativo, con capturas de las pantallas principales. Dos o tres capturas bien elegidas valen más que ocho: la consulta de disponibilidad, la reserva y los reportes.*

[Resumen de lo implementado.]

7.2. Cumplimiento de los criterios de aceptación

Instrucción. *Los seis criterios del documento de alcance, uno por fila, con la evidencia que respalda cada «Sí». Es la tabla que un evaluador busca primero. Si alguno no se cumple, decláralo: un criterio marcado como cumplido que luego falla en la demo cuesta más que uno declarado pendiente.*

Tabla 10: Cumplimiento de los criterios de aceptación.

#	Criterio	Estado	Evidencia
CA-1	[Consultar, crear y cancelar reservas de las tres canchas]	[...]	[...]
CA-2	[El administrador gestiona el catálogo y cancela cualquier reserva]	[...]	[...]
CA-3	[El sistema impide reservar un bloque ocupado]	[...]	[...]
CA-4	[Los reportes muestran ocupación y reservas por período]	[...]	[...]
CA-5	[Shell y al menos dos remotes integrados con Module Federation]	[...]	[...]
CA-6	[Al menos dos microservicios independientes sobre PostgreSQL]	[...]	[...]

7.3. Verificación de las reglas de negocio

Instrucción. *Cómo se comprobó que cada regla funciona. No hace falta un capítulo de pruebas: una tabla con el escenario, el resultado esperado y el obtenido es suficiente y se lee en treinta segundos. La colección de peticiones va al anexo.*

Tabla 11: Comprobación de las reglas de negocio.

Regla	Escenario	Resultado
[...]	[...]	[...]

7.4. Limitaciones

Instrucción. *Qué no se alcanzó y por qué. Declararlo suma: demuestra que se conoce el propio sistema. Omitir una limitación que el evaluador descubre en la demo resta el doble.*

[Limitaciones conocidas.]

8. Conclusiones

8.1. Lecciones aprendidas

8.2. Trabajo futuro

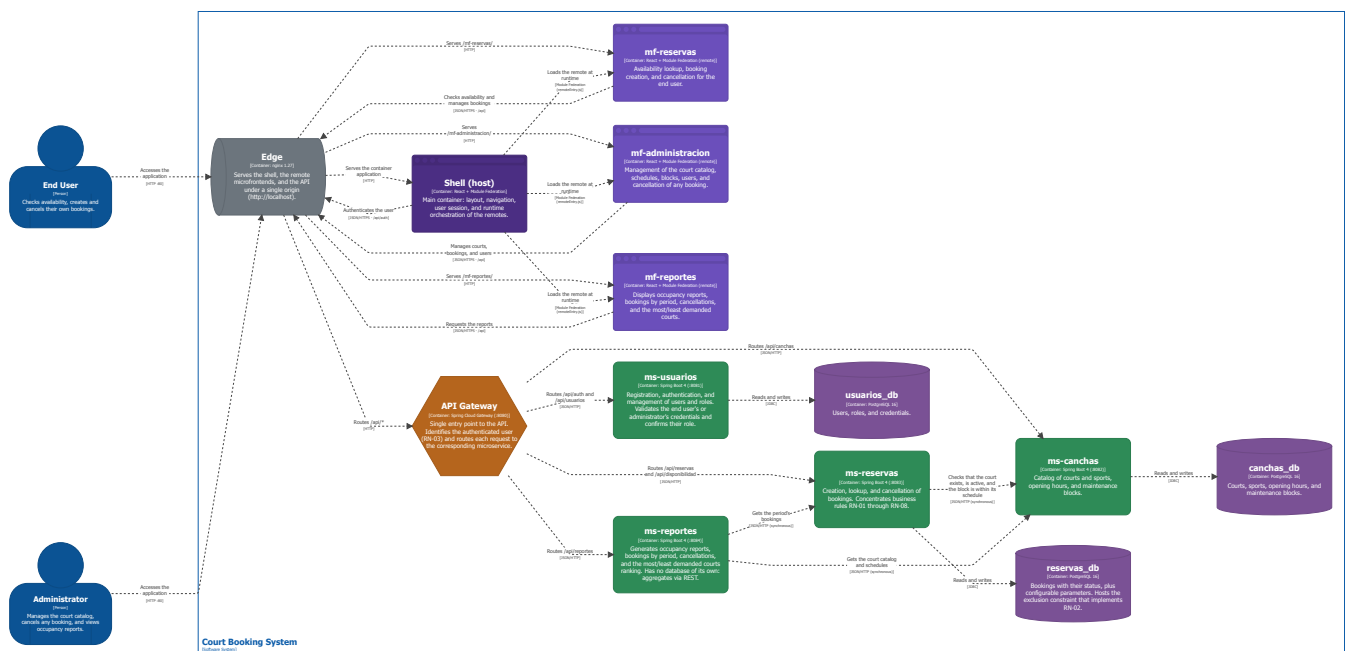


Figura 2: Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.

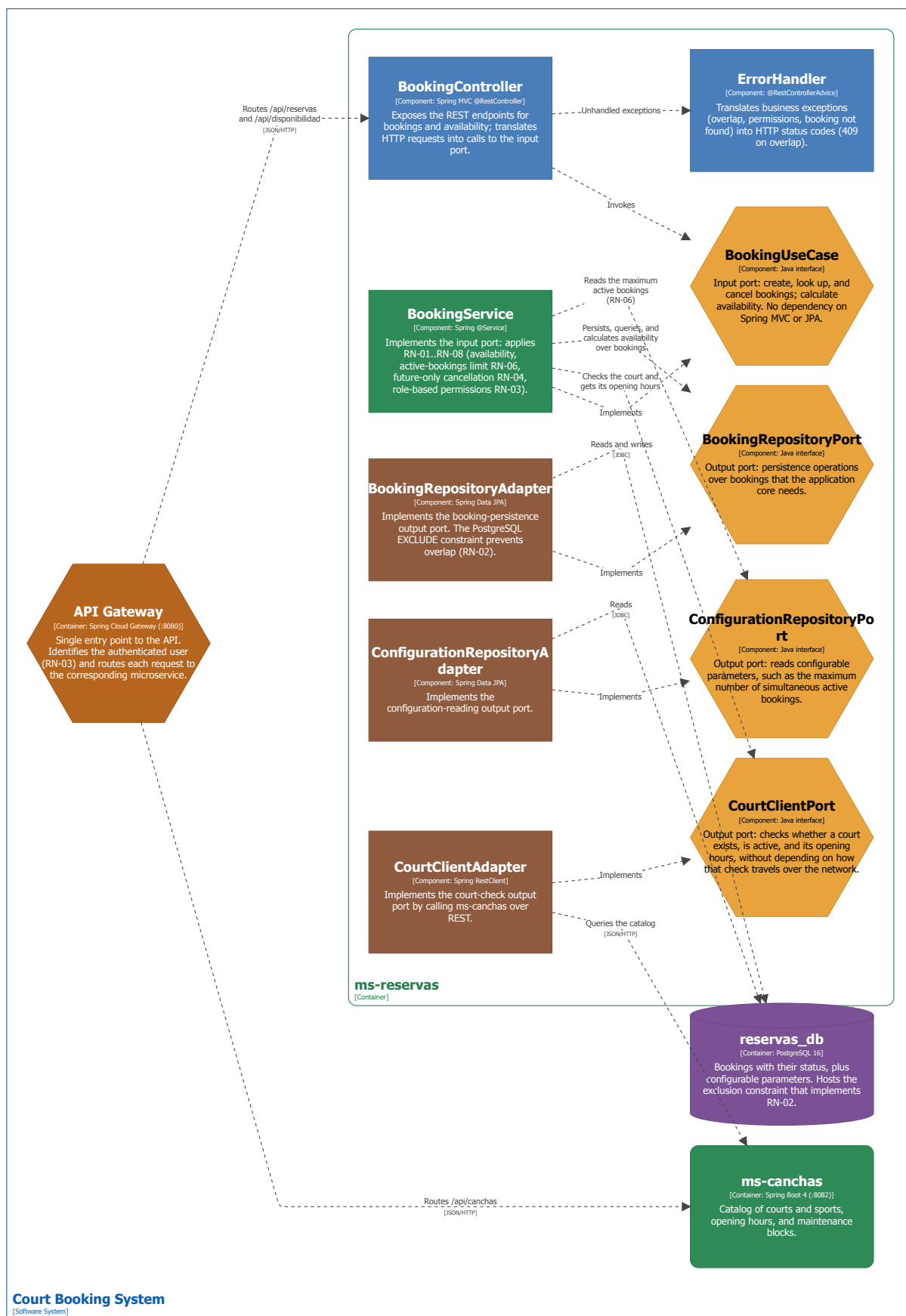


Figura 3: Vista de componentes de ms-reservas.

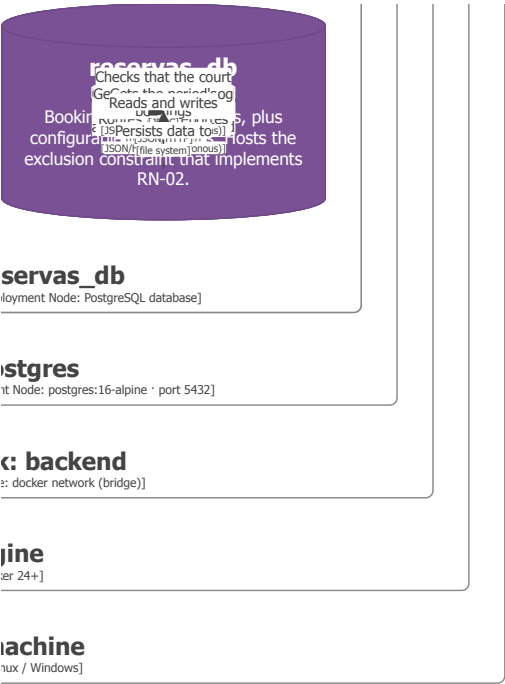


Figura 4: Vista de despliegue: contenedores Docker, redes y volumen.

Referencias

- [1] Organización. «Título de la página o documentación,» visitado 1 de ene. de 2026.
dirección: <https://ejemplo.org/documentacion>

A. Scripts de base de datos

B. Colección de pruebas de la API

C. Capturas de la aplicación

D. Repositorio del proyecto